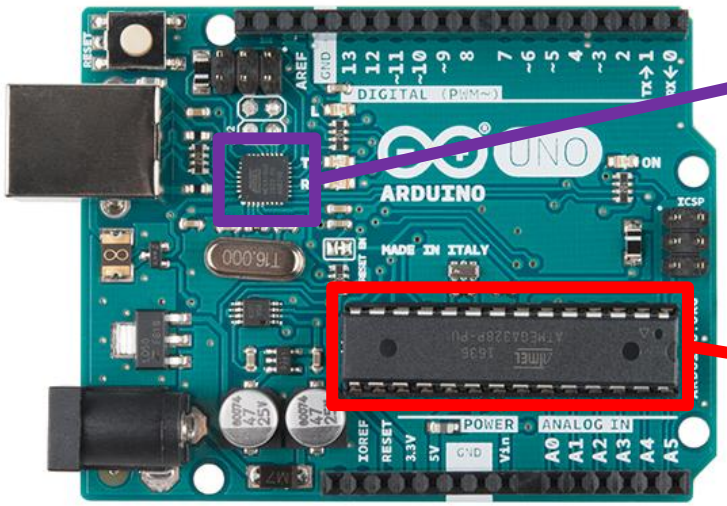
ECE0202 Embedded Processors & Interfacing + Lab

Sichuan University-Pittsburgh Institute

Development Boards

- The following slides will cover many development boards as well as the processors that are used on them

Arduino Uno R3

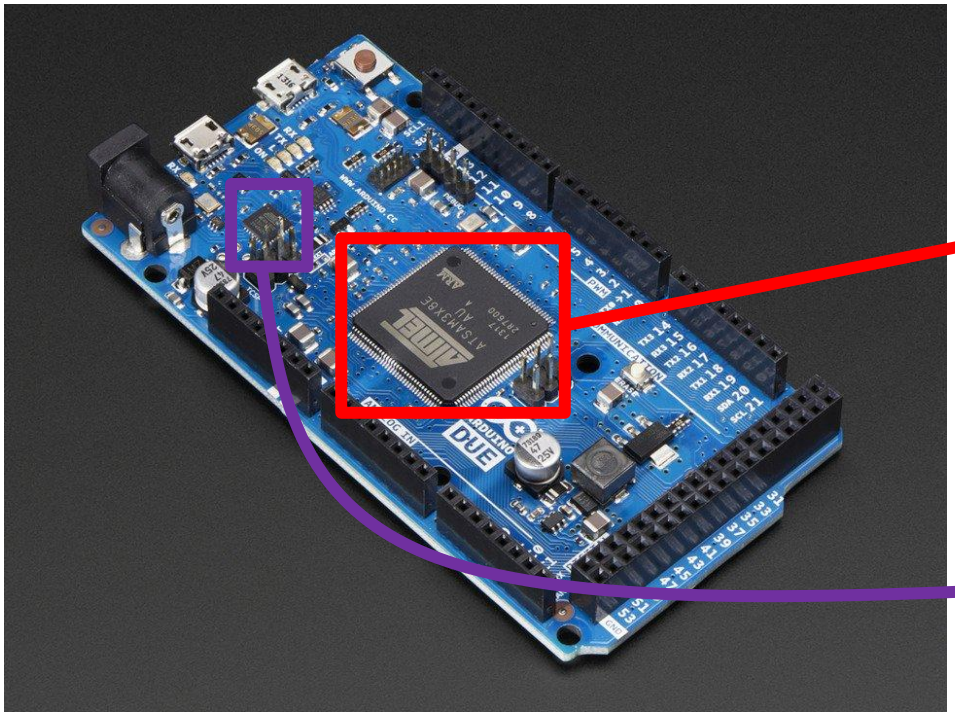


Atmel ATMega16U2 (8-bit)

Atmel ATMega328P (8-bit)

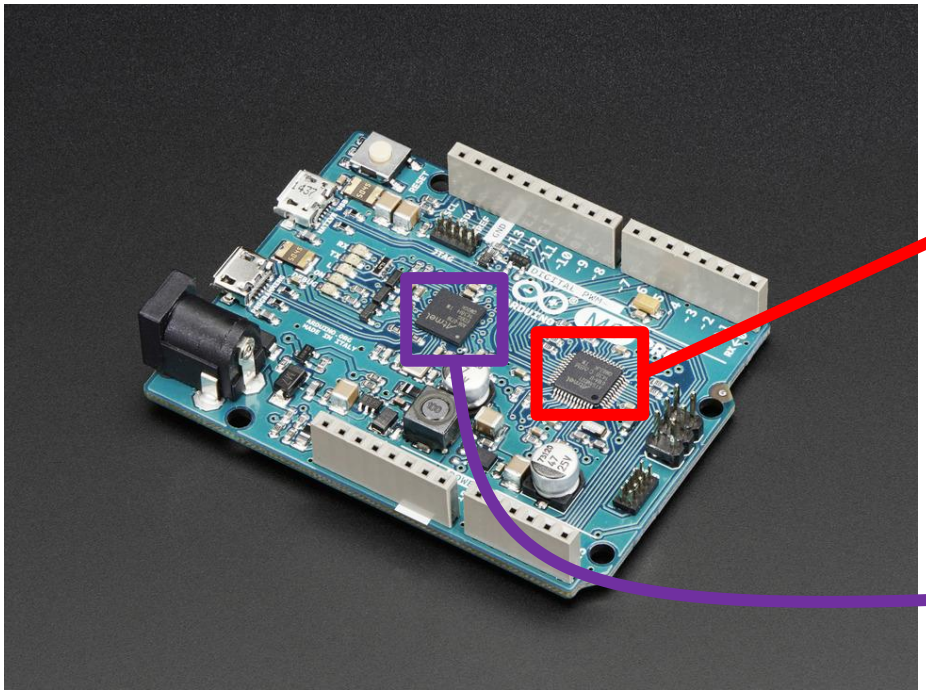
From https://www.sparkfun.com/products/11021

Arduino Due



From https://www.adafruit.com/product/1076

Arduino Mo

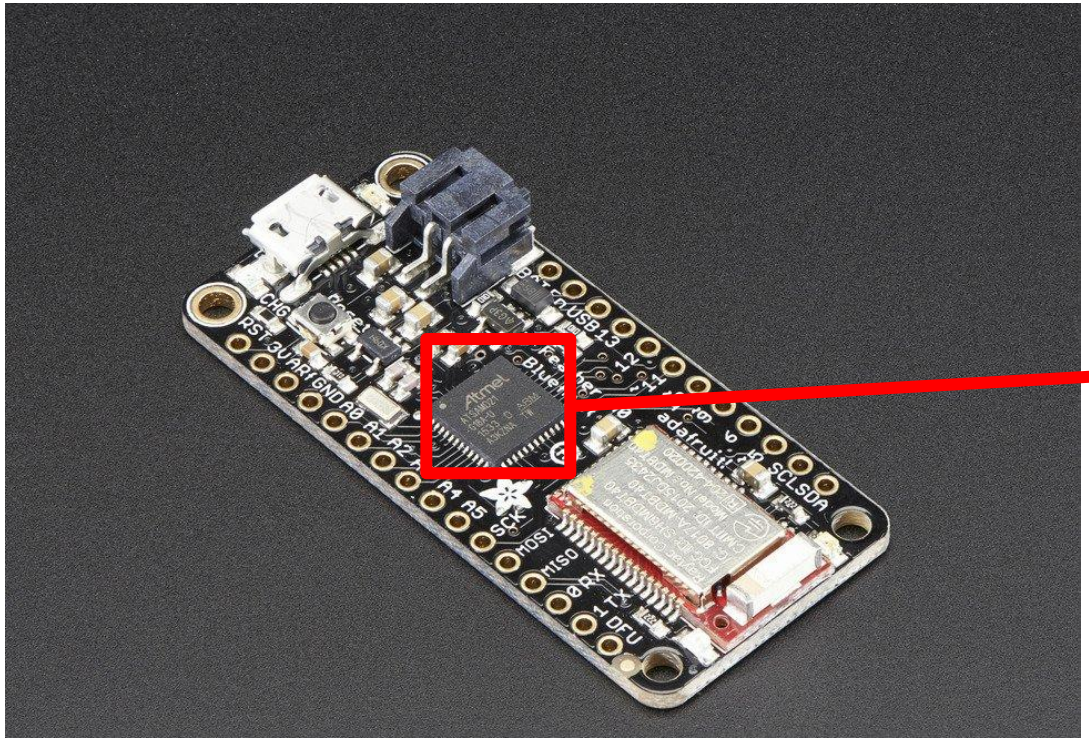


Atmel ATSAMD21
ARM Cortex-M0
(32-bit)

Atmel EDBG
Embedded
Debugger

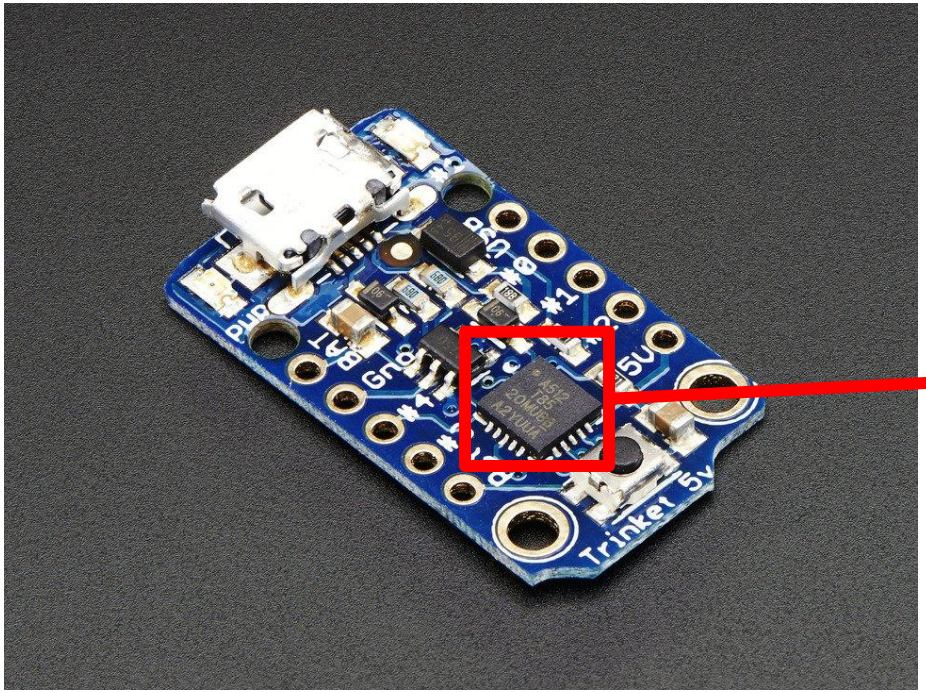
From https://www.adafruit.com/product/2417

Adafruit Feather M0



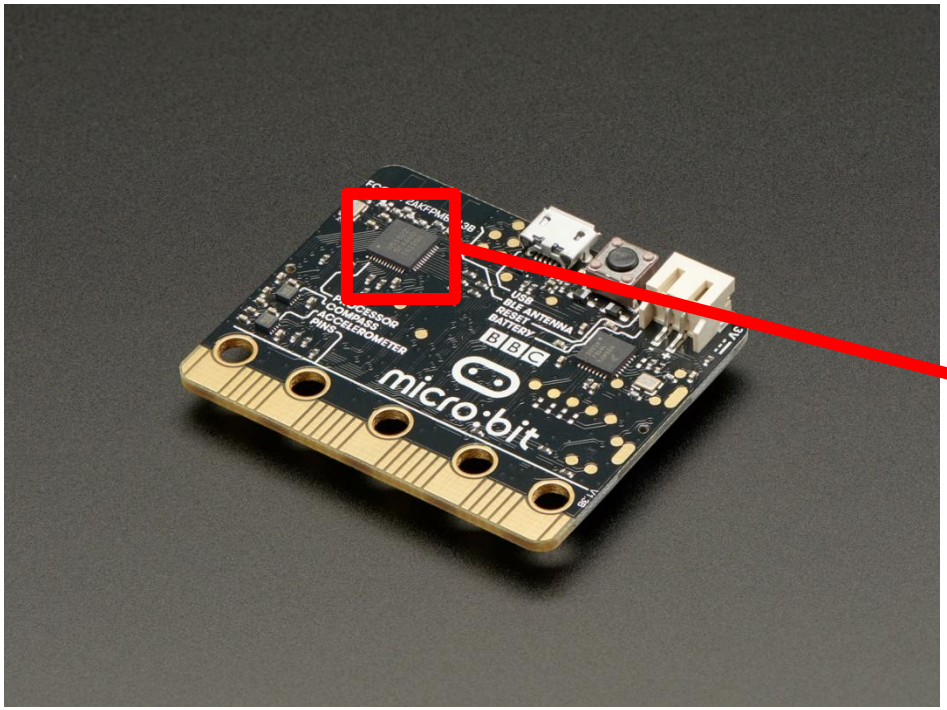
From https://www.adafruit.com/product/2995

Adafruit Trinket



From https://www.adafruit.com/product/1500

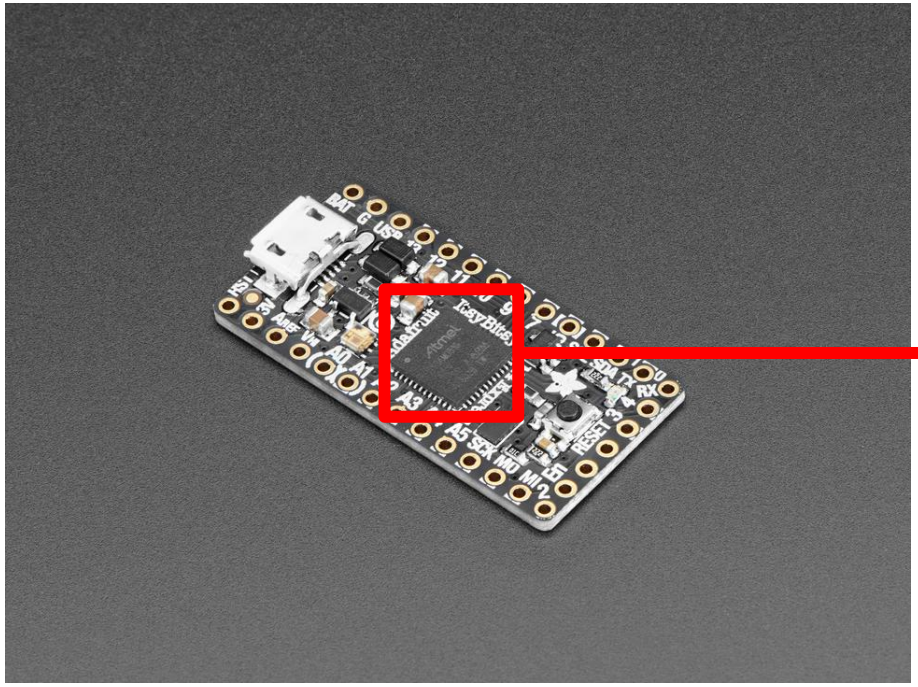
BBC micro:bit



Nordic Semiconductor nRF51822 ARM Cortex M0 (32-bit)

From https://www.adafruit.com/product/3530

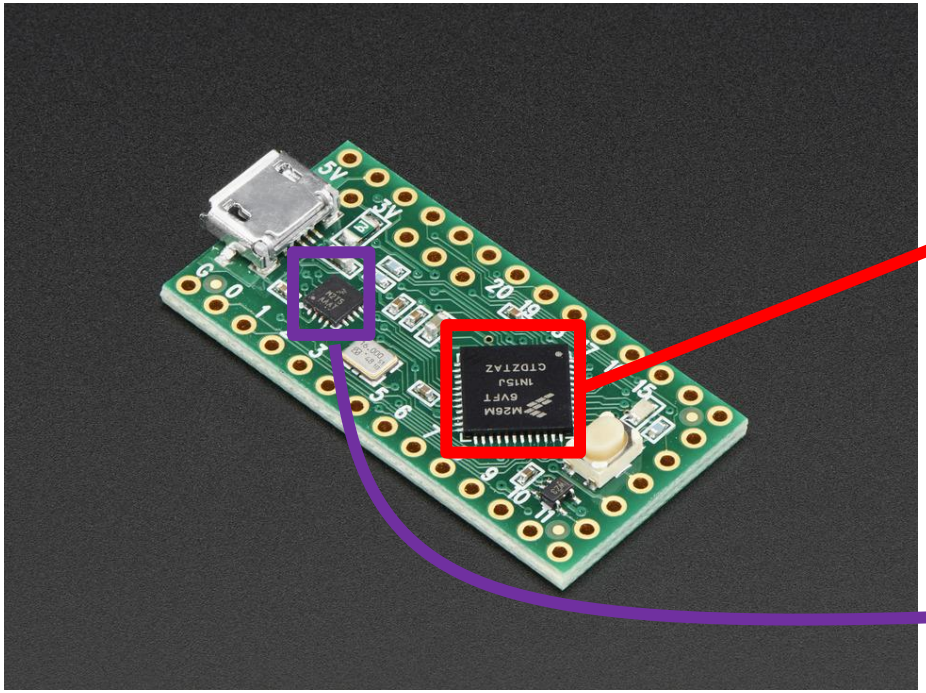
Adafruit ItsyBitsy M4 Express



Atmel ATSAMD51 ARM Cortex M4 (32-bit)

From https://www.adafruit.com/product/3800

Teensy-LC

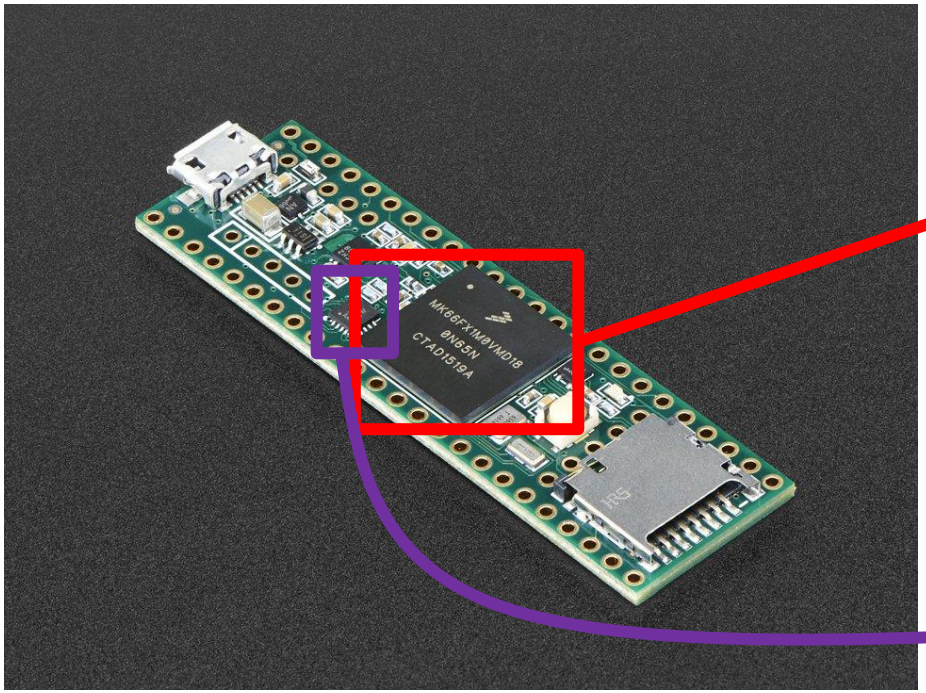


Freescale MKL26Z64VFT4 ARM Cortex-M0+ (32-bit)

Freescale Kinetis KL02 ARM Cortex M0 (32-bit)

From https://www.adafruit.com/product/2419

Teensy 3.6

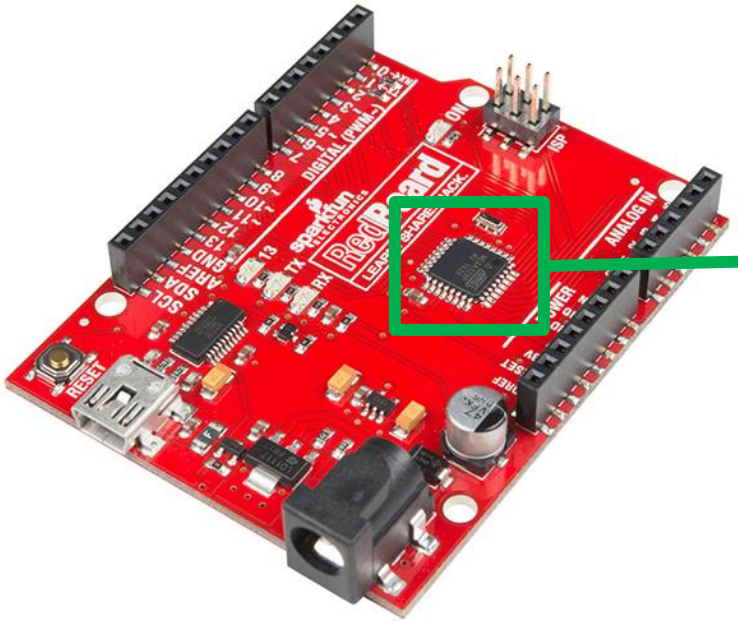


Freescale MK66FX1M0VMD 18 ARM Cortex-M4 (32-bit)

Freescale Kinetis KL02 ARM Cortex M0 (32-bit)

From https://www.adafruit.com/product/3266

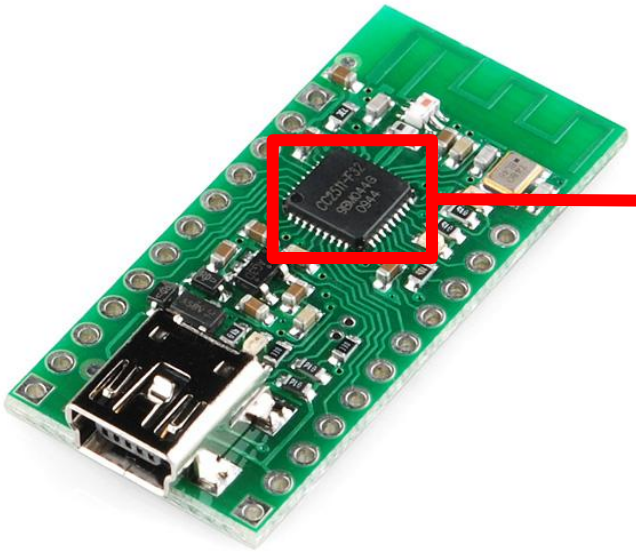
SparkFun RedBoard



Atmel ATMega328P (8-bit)

From https://www.sparkfun.com/products/13975

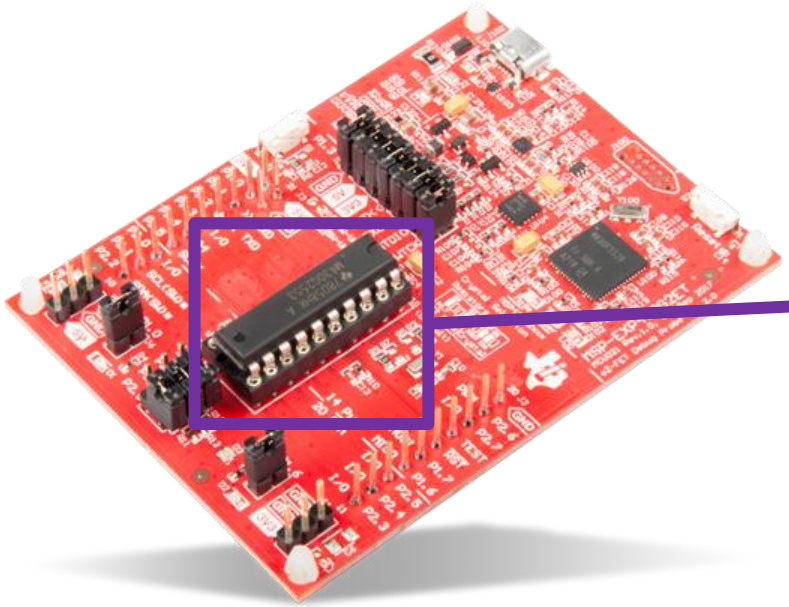
Pololu Wixel



Texas Instruments CC2511 based on 8051 core (8-bit)

From https://www.pololu.com/product/1336

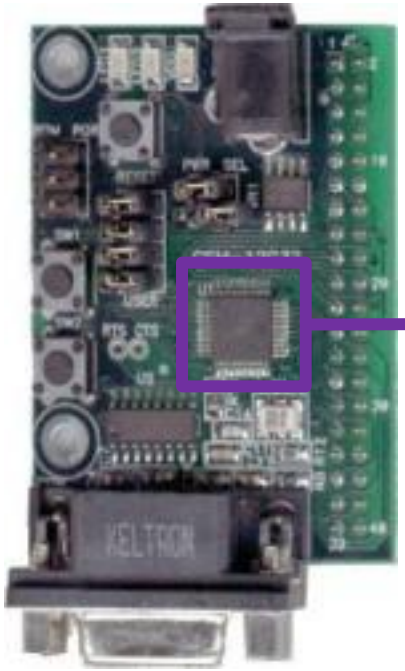
TI MSP430 Launchpad



Texas Instruments
MSP430 (16-bit)

From https://www.mouser.com/new/Texas-Instruments/ti-msp-exp430g2et-dev-kit/

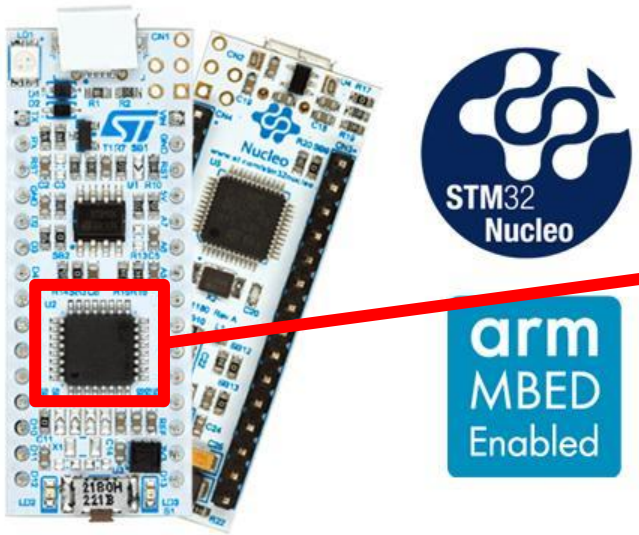
Axiom Manufacturing CSM-12C32



Freescale MC9S12C32 based on MC68HC12 core (8-bit)

From https://axman.com/content/csm-12c32-low-cost-evaluation-module-freescale-mc9s12c32

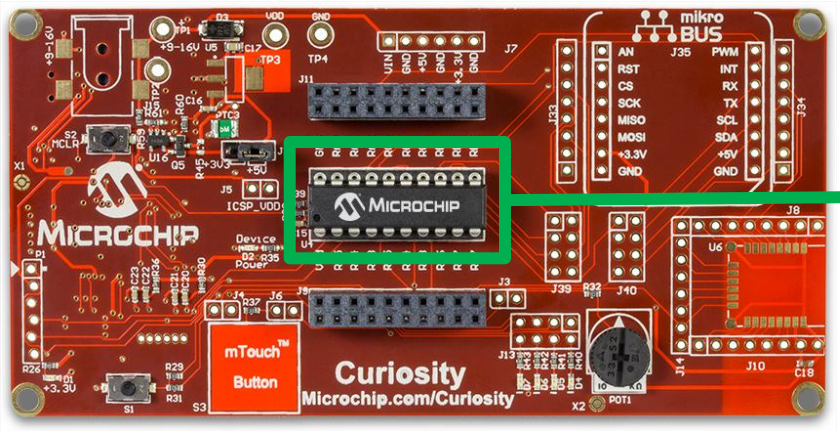
ST NucleoF303



ST Micro
STM32F303K8
ARM Cortex M4
(32-bit)

Form https://www.st.com/content/st_com/en/products/evaluation-tools/product-evaluation-tools/mcu-mpu-eval-tools/stm32-mcu-mpu-eval-tools/stm32-nucleo-boards/nucleo-f303k8.html

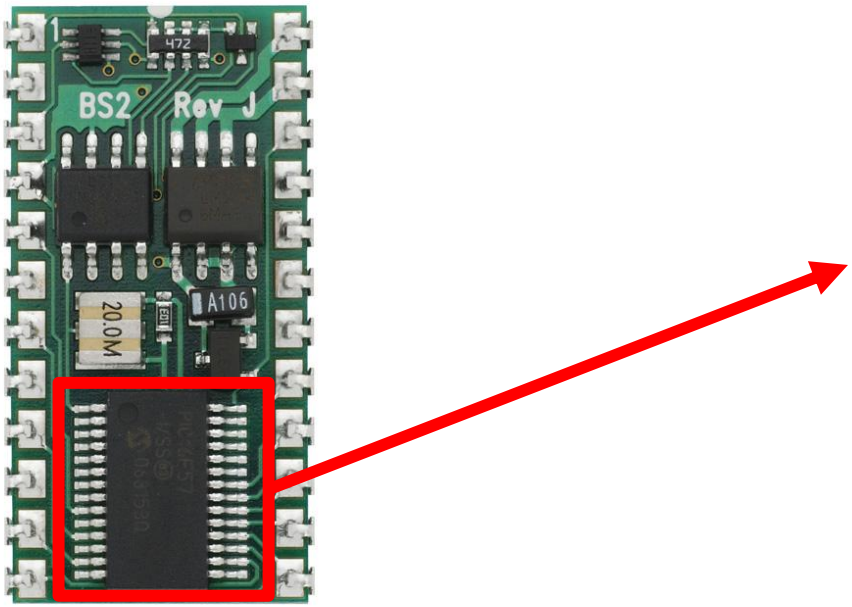
Microchip Curiosity Board



Various PIC microcontrollers such as 16F84a (8-bit)

From https://www.microchip.com/DevelopmentTools/ProductDetails/PartNO/DM164137

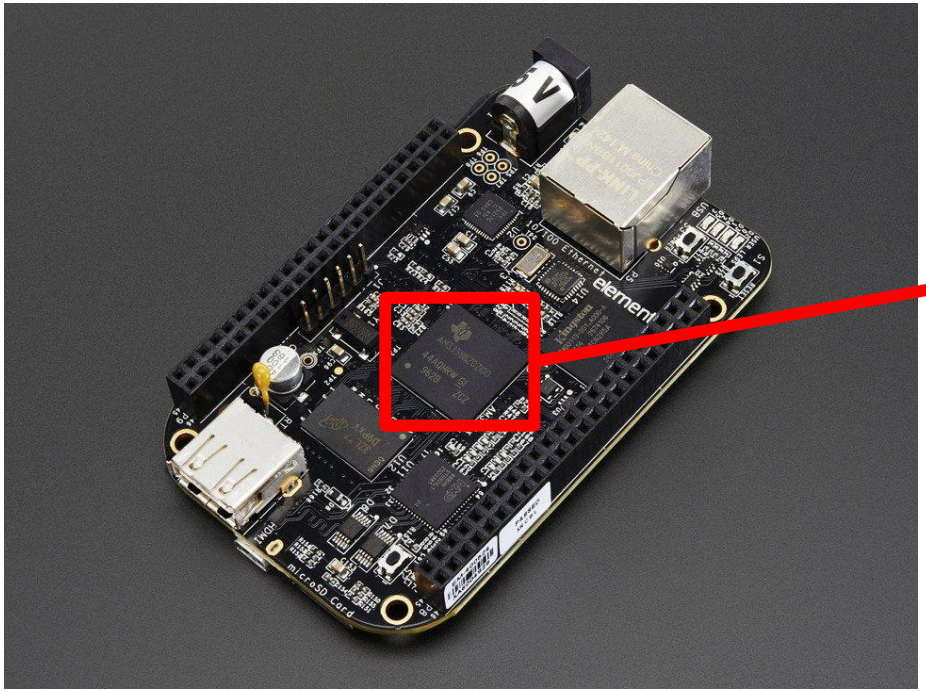
Parallax BASIC Stamp 2



Microchip
PIC16C57C (8-bit)

From https://www.pololu.com/product/1600

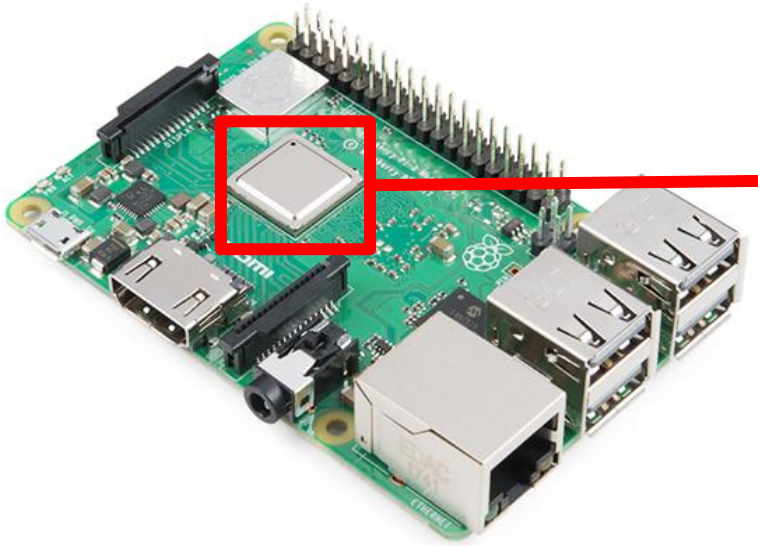
BeagleBone Black



Texas Instruments
AM3358 Sitara
ARM Cortex A8
(32-bit)

From https://www.adafruit.com/product/1996

Raspberry Pi 3 B+



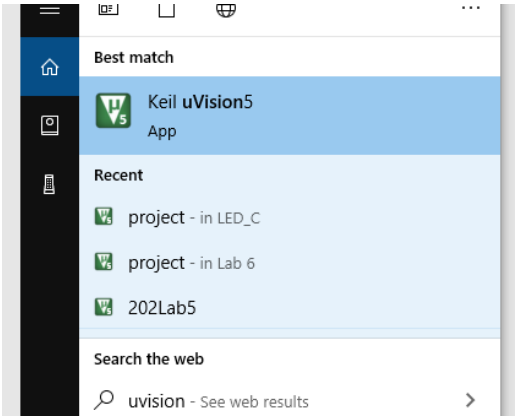
Broadcom BCM2837B0
ARM Cortex A57 (64-bit)

From https://www.sparkfun.com/products/14643

Software: Keil uVision5

KEIL uVision

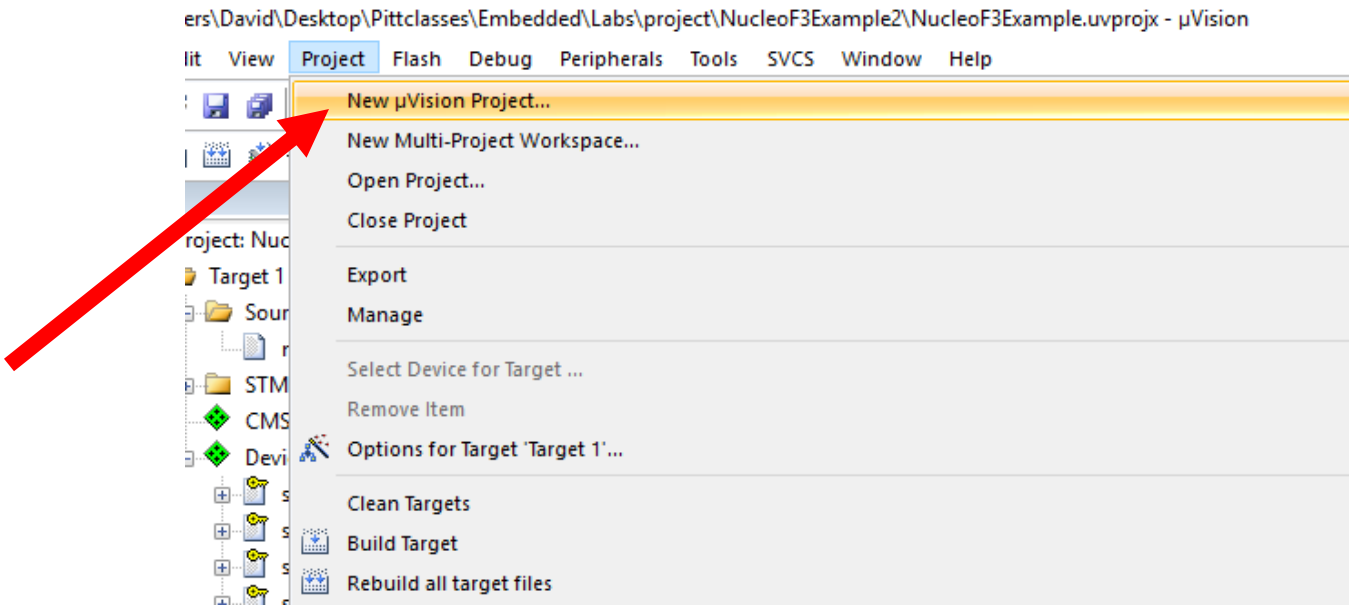
- Integrated Development Environment (IDE) for ST ARM embedded processors
- Allows for coding, programming, and real-time debug



Project Creation

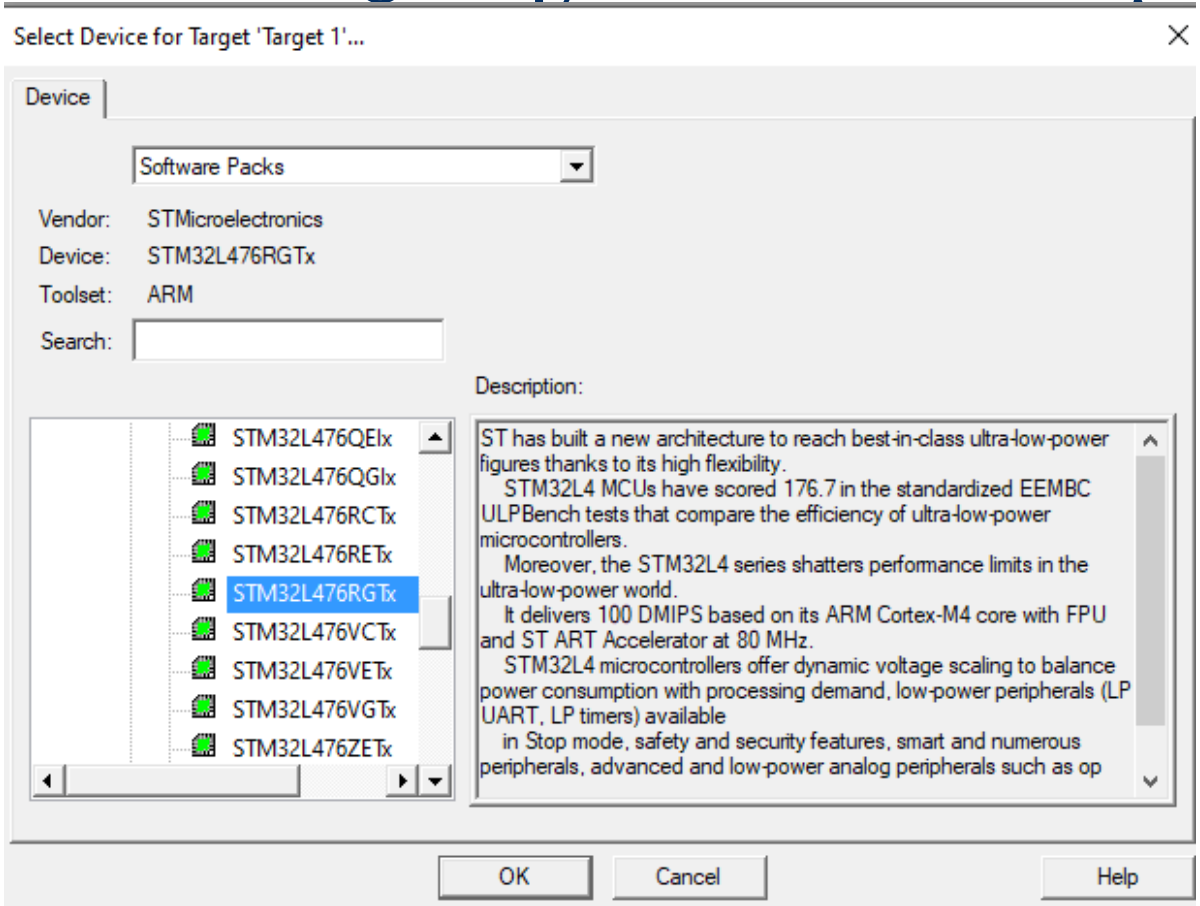
KEIL Project Creation (Assembly)

- Click on Project -> New uVision Project



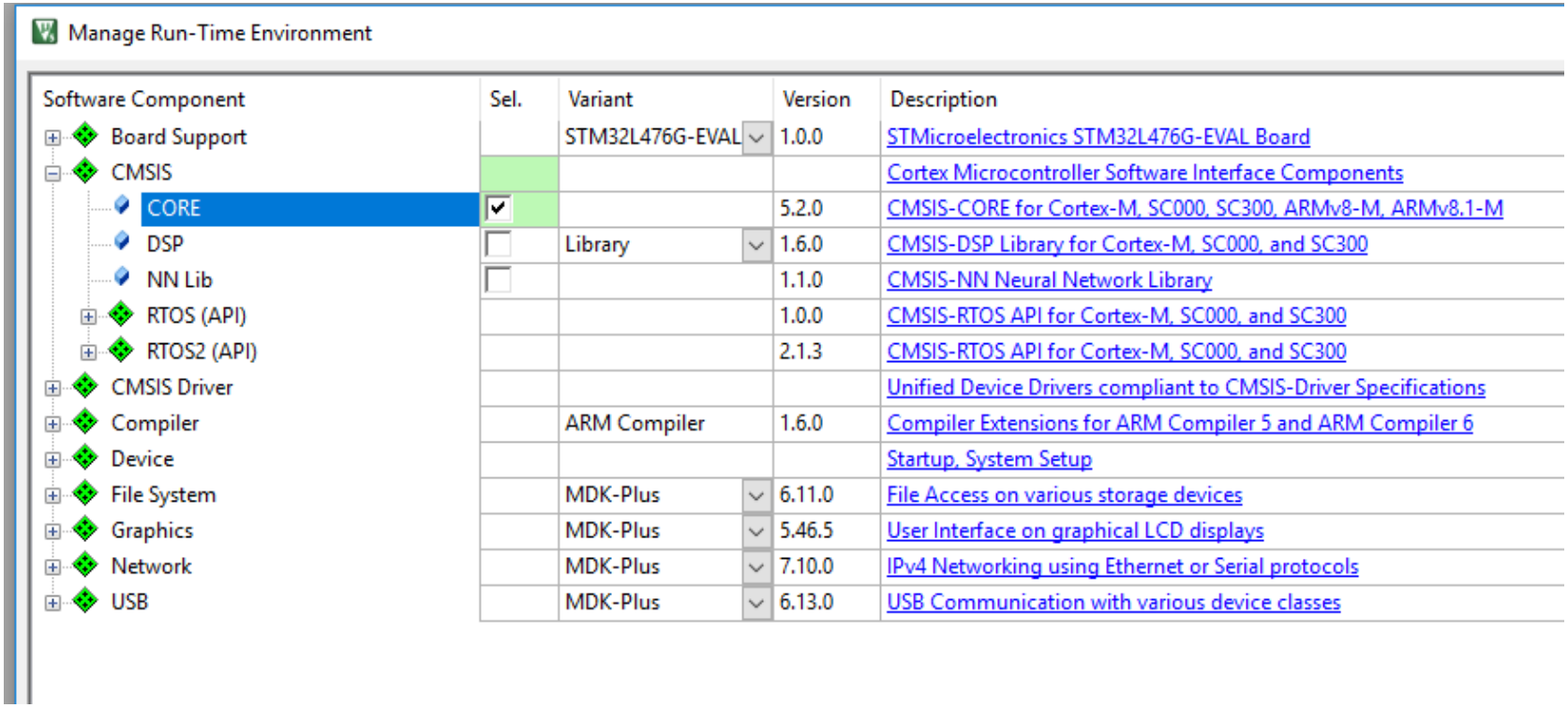
KEIL Project Creation

- Select STM32L476RGTx as the processor



KEIL Project Creation

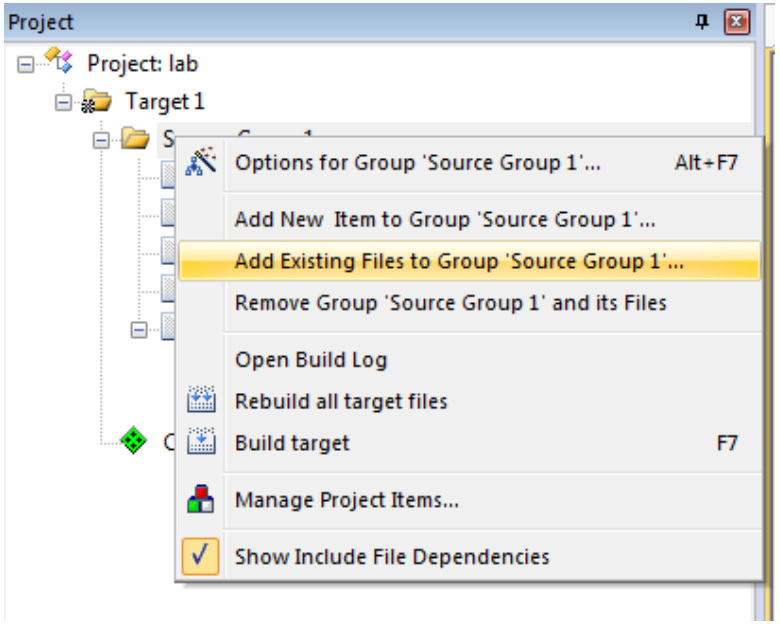
- For the Run-Time Environment, select the checkbox for CMSIS-CORE



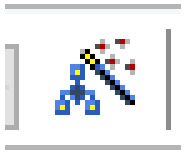
KEIL Project Creation

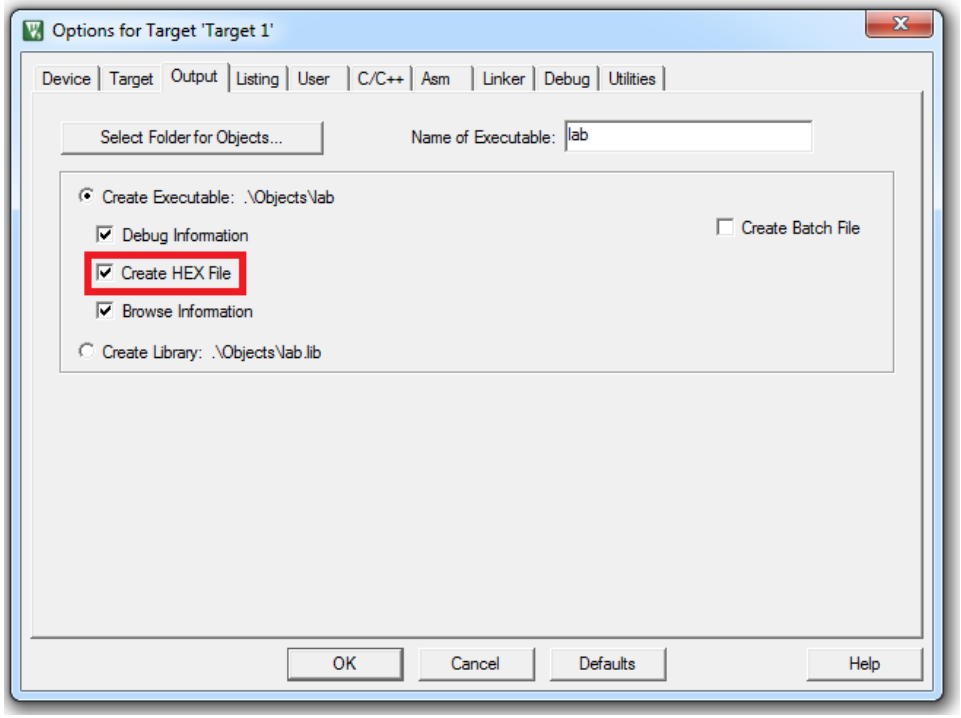
- To create a project, you should download all the following files and include them in the project. Right click on the folder “Target 1” and add the following files:

  - **startup_stm32l476xx.s**
  - **main.s**
  - core_cm4_constrants.s
  - stm32l476xx_constants.s
  - SysClock.c
  - UART.c



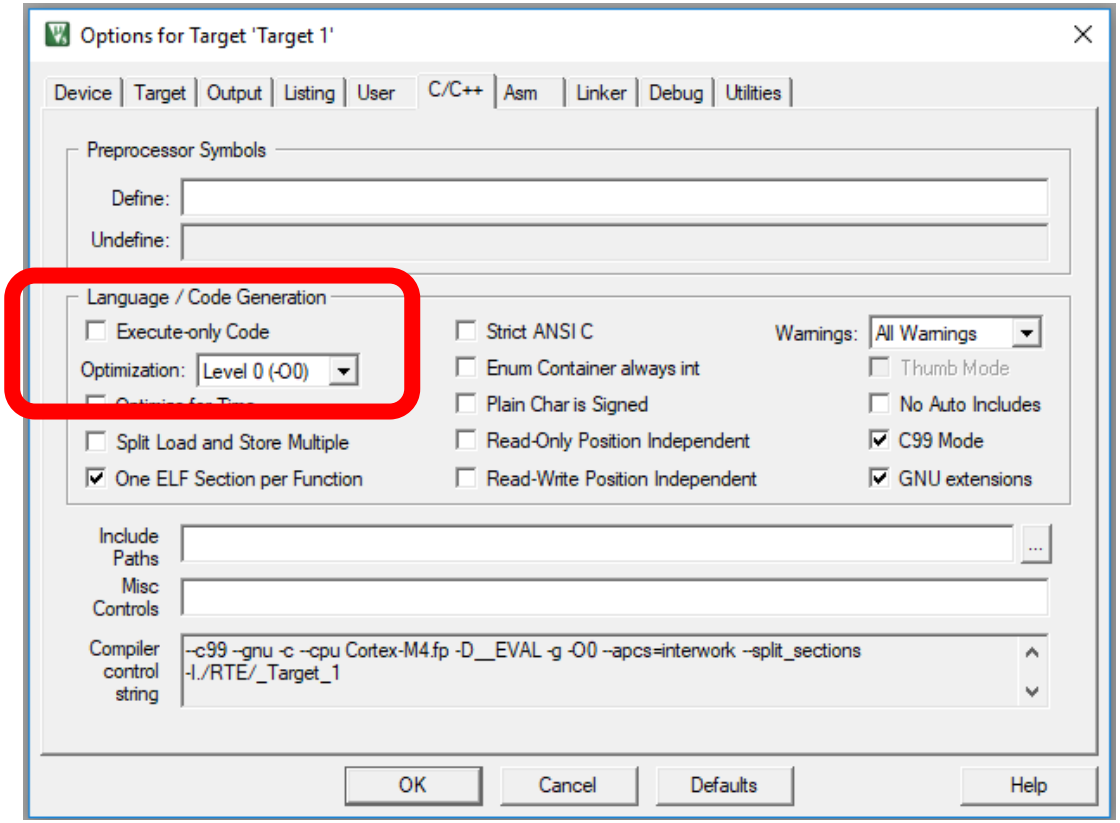
KEIL Project – Target Properties

- Click on Target Options 
- Click “Output” and Select “Create HEX File”



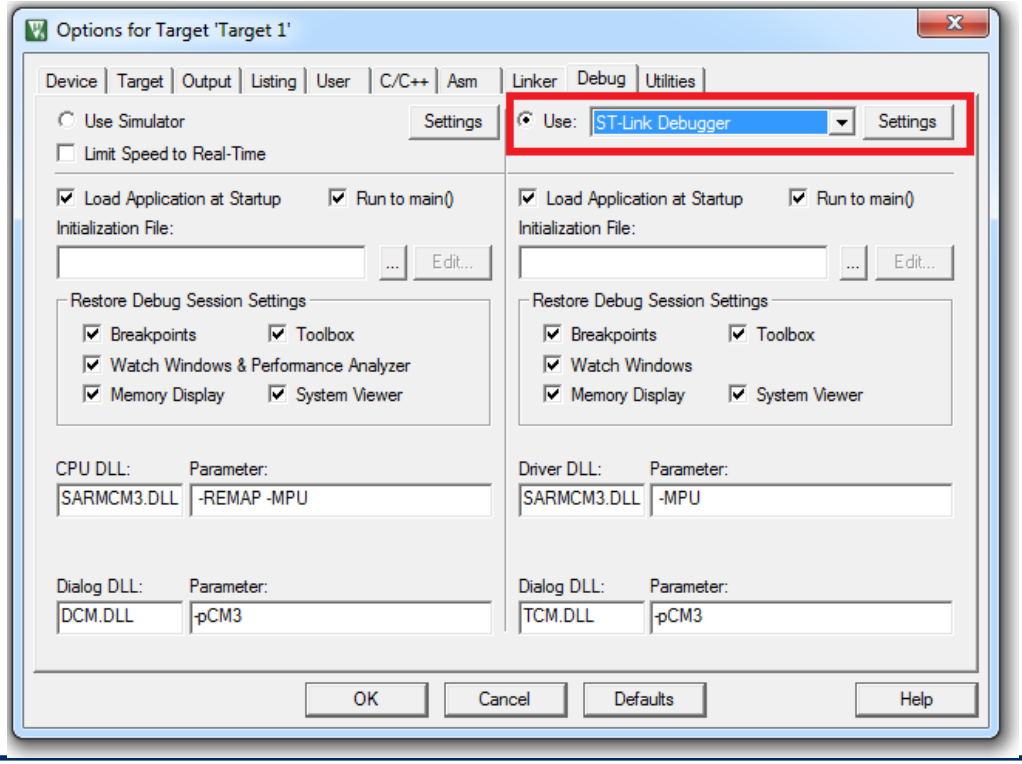
KEIL Project – Target Properties

Ensure that Optimization is set to “Level 0” – this is important for C projects



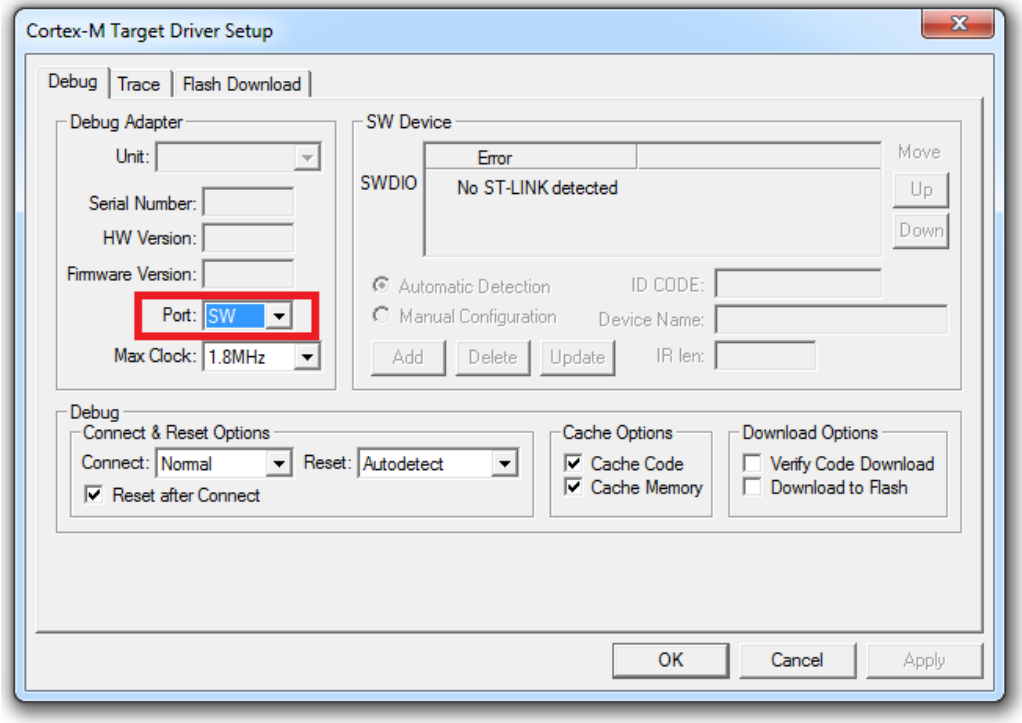
KEIL Project – Target Properties

- On the debug page, select to use “ST-Link Debugger”. This tells KEIL that you will debug code in real-time as it runs on your board.

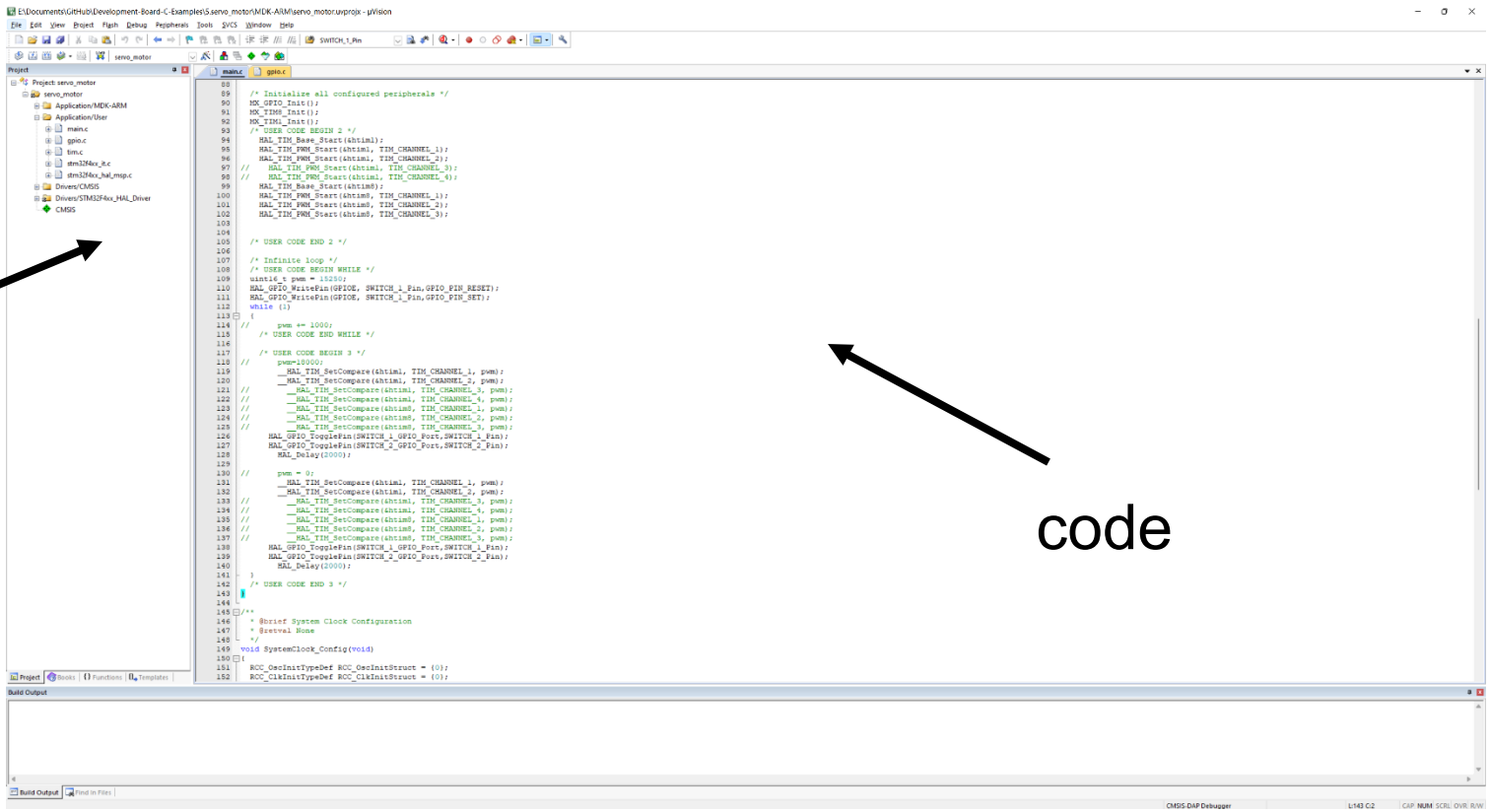


KEIL Project – Target Properties

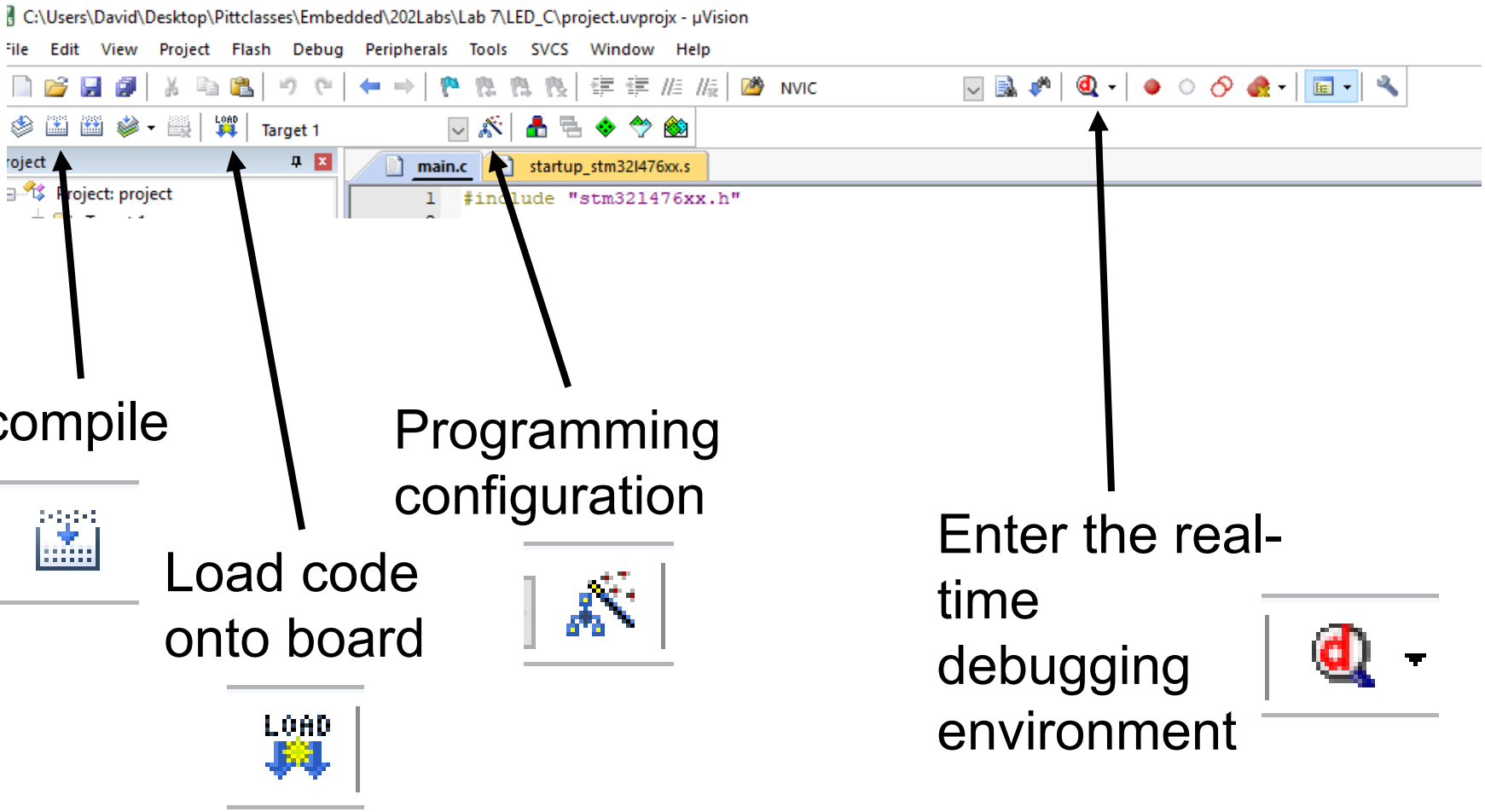
- On the Settings Page, select Serial Wire "SW" as the port



KEIL main window



KEIL toolbar

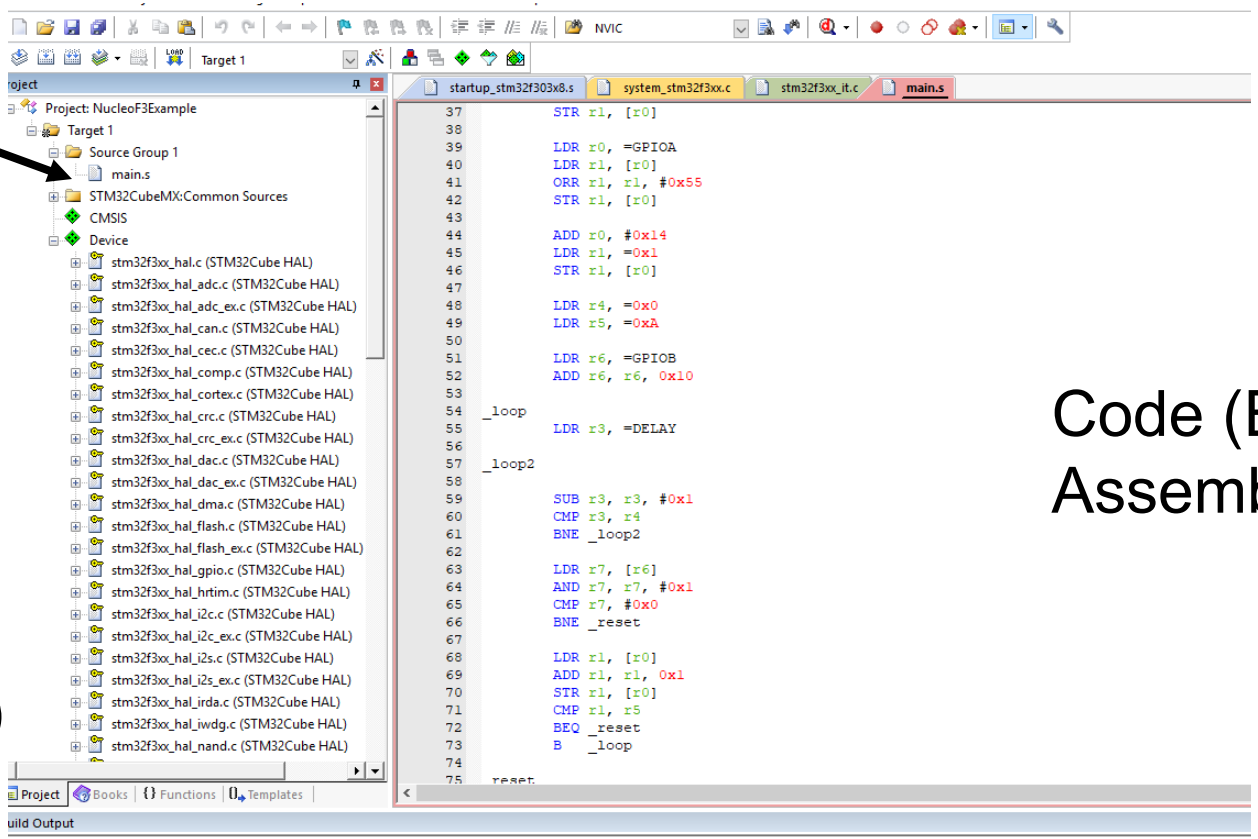


Writing Code

- Assembly source files will end with .s
- C source files will end with .c or .h (code or header)

Assembly Project

Main file

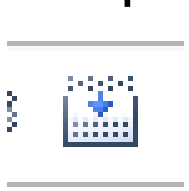


Extra Included Files (Depends on Project)

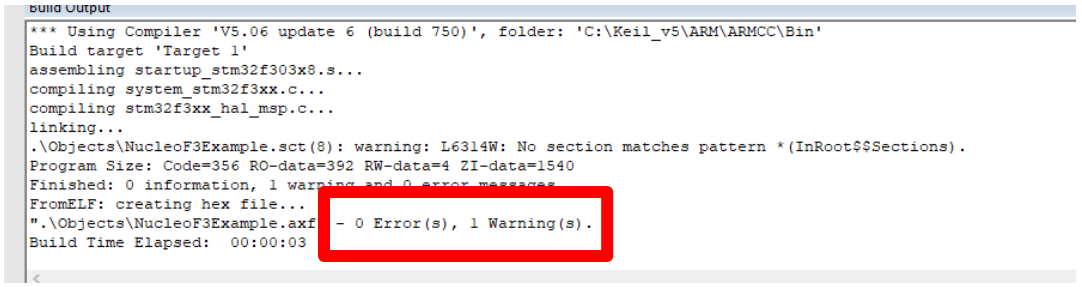
Code (Either Assembly or C)

Building your Code

- Code must be built – translated from C or assembly into "machine code" that is loaded directly into the processor memory



- If you receive "Warnings" during the build stage this is generally OK – if you compile again the warnings will disappear

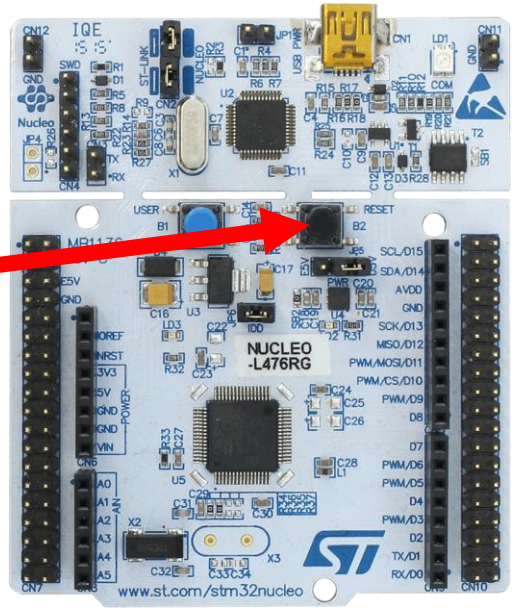


Loading your code onto the board

- Press the load button to load code onto the STM32 Discovery board

Load code onto board

- You have to press the reset button on the Discovery board before the code will actually run

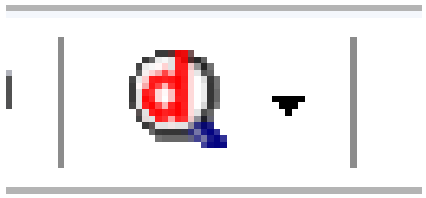
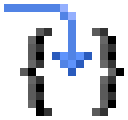
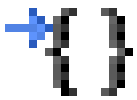


Debug

Running Code with Debug

- Debug allows you to run code one line at a time, or stop at certain points using breakpoints
- You should ALWAYS debug your code before running it in real-time
- It is best to approach labs in steps
  - Write code for a small part, debug
  - Write code for next small part, debug

Running Code with Debug

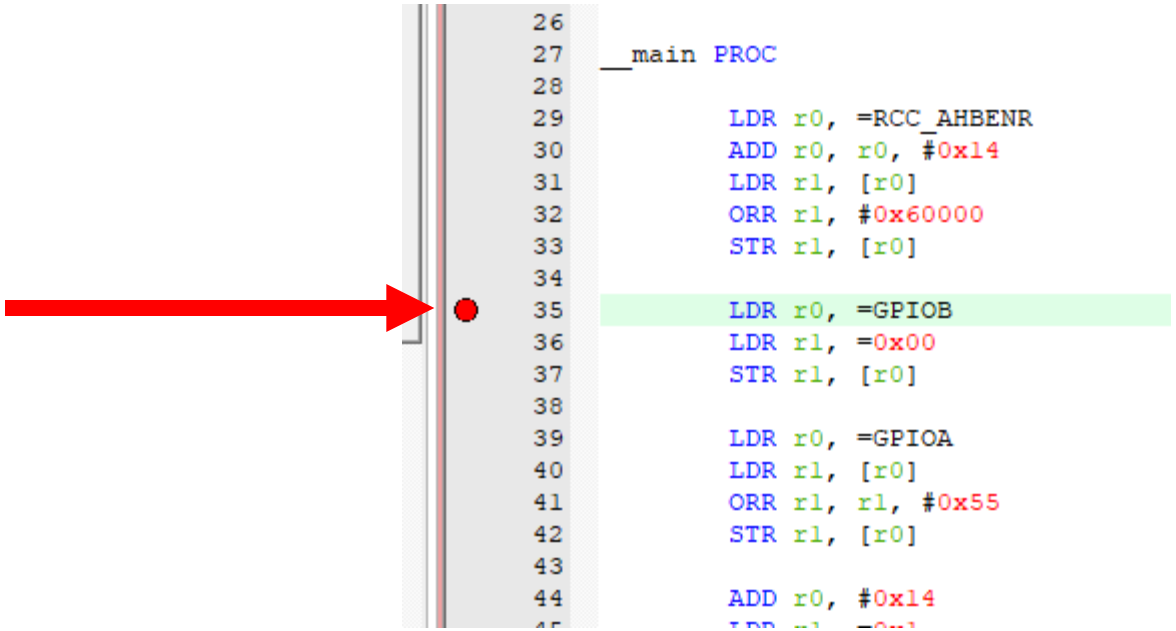
- Enter Debug Mode 
- Step through code one line at a time 
- Run to next breakpoint 

Breakpoints

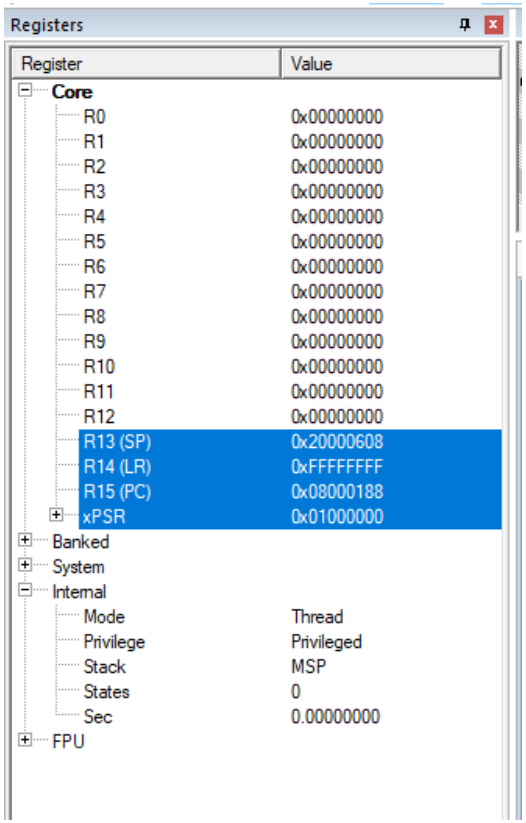
- Assembly and C are sequential code
- Breakpoints allow you to stop the processor operation at a certain line of code
- You can check register contents, flag statuses, variable values, etc

Breakpoints

- A breakpoint is set by clicking on the gray vertical bar to the left of any line of code
- A red dot will appear to indicate a breakpoint

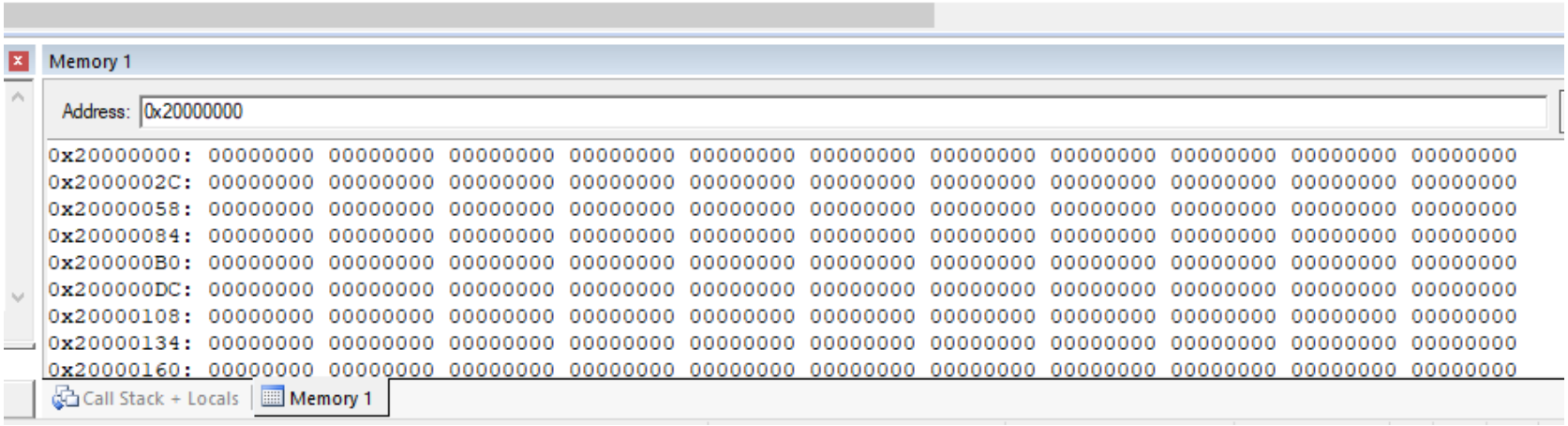


Debug Core Watch Window



- Allows you to view current core register contents, status flags, link register, program counter, stack pointer
- All of this will be explained in the lectures coming up!

Debug Memory Watch Window



- Allows you to view current data memory contents
- All of this will be explained in the lectures coming up!